卡片式对话的协议方案探索和思考

原创 跨端技术团队 大淘宝技术 2026年4月17日 01:01 浙江 3人

本文探讨了在智能助手对话流中实现卡片式交互的系统性工程方案。文章围绕卡片如何嵌入Markdown流、数据来源以及多团队协作规范三个核心问题，详细分析了代码块扩展、占位符替换及自定义标签等嵌入方案，并对比了模型直出、增量Patch及Tool驱动等数据获取模式。最终，文章提出了一套包含Markdown标记、消息传输、UI渲染及事件通信的四层统一协议体系，旨在解决Agent时代下多端一致性、数据实时性及跨团队协作混乱的问题，为构建高效、标准化的卡片式对话系统提供了落地的架构参考。

01 引言

当我们第一次在智能助手中看到一张可交互的商品卡片时，觉得这不过是"在聊天框里塞了个组件"。但真正实践下来才发现，这件事远比想象中复杂——它不是一个前端渲染问题，而是一个贯穿 Agent 设计、 模型输出、数据流转、协议设计、跨端一致性的系统工程。

这篇文章不讲概念，讲我们探索过程中踩过的坑和沉淀的方案。

文章将围绕三个问题展开：

1. **卡片如何嵌入对话流？** ——模型输出的是 Markdown 文本流，卡片怎么"混"进去？
2. **卡片数据从何而来？** ——模型不知道实时价格，数据该由谁提供、怎么填充？
3. **多团队协作怎么不乱？** ——当 N 个业务方都要接卡片时，如何用协议约束混乱？

02 卡片如何嵌入对话流

问题的本质

大模型的输出本质上是一个 token 流，经过拼接后形成 Markdown 文本。Markdown 本身是一种排版标记语言，它能表达标题、列表、代码块、图片，但无法表达"这里应该渲染一个商

品卡片组件"这件事。

所以核心问题是：如何在不破坏 Markdown 流式解析的前提下，嵌入自定义 UI 组件的语义信息？
我们调研和实践了三种方案，各有适用场景。

方案一：代码块扩展

这是我们最终在生产环境中采用的方案，原理非常直觉——Markdown 的代码块本身就有一个 `language` 字段（用于语法高亮），我们把这个字段"借用"为组件类型标识。
模型输出的 Markdown 长这样：

````
为你推荐以下商品：
```ProductCard
{
  "title": "iPhone 15 Pro Max",
  "description": "全新钛金属设计，A17 Pro 芯片",
  "itemPrice": 9999,
  "imageUrl": "https://example.com/iphone.jpg",
  "discount": "限时优惠 -10%"
}
```
以上商品支持分期免息，点击卡片可查看详情。
````

前端 Markdown 渲染器拿到这段文本后，在解析代码块时检查 `language` 字段：如果是 `javascript`、`python` 这种已知语言就做语法高亮；如果是 `ProductCard` 这种自定义标识，就把代码体当作 JSON 解析，传给对应的 React 组件渲染。
以 `react-markdown` 为例，扩展逻辑如下：

```
import Markdown from 'react-markdown';
import { ProductCard } from './components/ProductCard';
// 组件注册表：language 名 → React 组件的映射
const CARD_COMPONENTS = {
  ProductCard: ProductCard,
  UserProfile: UserProfile,
  FlightCard: FlightCard,
};
function ChatMessage({ markdown }) {
  return (
    <Markdown
      children={markdown}
```

```
      components={{
        code(props) {
          const { children, className, ...rest } = props;
          // className 格式为 "language-xxx"，提取 xxx 部分
          const match = /language-(\w+)/.exec(className || '');
          const langName = match ? match[1] : null;
          // 命中组件注册表 → 渲染业务卡片
          if (langName && CARD_COMPONENTS[langName]) {
            const CardComponent = CARD_COMPONENTS[langName];
            const cardProps = JSON.parse(String(children));
            return <CardComponent {...cardProps} />;
          }
          // 未命中 → 走默认的代码高亮渲染
          return (
            <code {...rest} className={className}>
              {children}
            </code>
          );
        },
      }}
    />
  );
}
```

这里有几个实践细节值得展开：

1. 为什么选代码块而不是其他 Markdown 元素？

代码块有两个天然优势：一是所有 Markdown 解析器都支持 language 字段的提取，不需要写自定义 parser；二是代码块内部的内容不会被 Markdown 解析器二次处理（不会把 `{` 当成列表、把 `"` 当成引号），JSON 数据可以安全传递。

2. 流式渲染的兼容性

代码块有明确的开始标记（```）和结束标记（```），流式解析器在遇到开始标记时就能知道"接下来是一个代码块"，在结束标记到来前持续缓冲内容。这意味着前端可以在代码块完整输出后再做 JSON 解析和组件渲染，不会出现半截 JSON 导致的 parse error。

3. 模型输出约束

大模型不会天然知道要输出 `ProductCard` 格式的代码块。我们通过 System Prompt 给模型一份"卡片生成规范"：

```
# 卡片生成规范
当需要展示结构化信息（商品、航班、用户资料等）时，使用 Markdown 代码块格式：
## 语法
- 语言标识使用组件名（PascalCase），如 `ProductCard`、`FlightCard`
```

```
- 代码体使用标准 JSON，属性名 camelCase
- 确保 JSON 格式合法，属性名使用双引号
## 示例
```ProductCard
{
  "title": "商品名称",
  "itemPrice": 99,
  "imageUrl": "https://example.com/img.jpg"
}
```

实际调试中发现，给模型 2-3 个 Few-Shot 示例就能稳定输出正确格式。如果对格式一致性要求极高，还可以在服务端加一层正则校验——代码块输出完毕后检查 JSON 是否合法，不合法就降级为纯文本展示。

方案二：占位符替换

占位符替换是一个更轻量的思路——模型在 Markdown 中输出一个特殊标记（比如 `__placeholder__ProductCard`），前端识别到后替换为对应组件。

```
推荐上午 10 点从杭州出发前往上海的高铁__placeholder__ProductCard,早班直达。
```

这个方案看起来简单，但在流式场景下有一个棘手的体验问题：模型是逐 token 输出的，当只输出到 `__place` 的时候，前端无法判断这是一个占位符还是普通文本。用户会先看到一串乱码般的占位符文字，然后突然跳变为卡片。

我们尝试过一种优化方式：把占位符设计成类似超链接的语法 `[(ProductCard)]`。利用 Markdown 解析器的已有机制——遇到 `[` 时暂存等待后续字符，如果后续不符合链接语法就回退为普通文本。在此基础上扩展：

```
遇到 [( → 暂存，推测可能是卡片
等待 )] 出现 → 匹配成功，渲染组件
超过 buffer 上限仍未匹配 → 回退为普通文本
```

在 `markdown-it` 中实现为一个 inline rule 插件：

```
function componentPlugin(md) {
  md.inline.ruler.before('link', 'component', (state, silent) => {
    const start = state.pos;
    const max = state.posMax;
```
```

```
    // 检查起始标记 [(
    if (state.src.charCodeAt(start) !== 0x5B /* [ */ ||
        state.src.charCodeAt(start + 1) !== 0x28 /* ( */) {
      return false;
    }
    // 向前扫描，寻找结束标记 )]
    let pos = start + 2;
    while (pos < max - 1) {
      if (state.src.charCodeAt(pos) === 0x29 /* ) */ &&
          state.src.charCodeAt(pos + 1) === 0x5D /* ] */) {
        break;
      }
      pos++;
    }
    if (pos >= max - 1) return false;
    const componentName = state.src.slice(start + 2, pos);
    if (!silent) {
      const token = state.push('component', '', 0);
      token.content = componentName;
    }
    state.pos = pos + 2;
    return true;
  });
  md.renderer.rules.component = (tokens, idx) => {
    const name = tokens[idx].content;
    return `<div class="custom-component" data-component="${name}"></d
  };
}
```

占位符方案的优势是改造成本低（不需要模型生成完整 JSON），适合卡片数据完全由服务端异步填充的场景。但劣势也很明显：占位符本身不携带数据，卡片要么需要事后补数据（增加一次请求），要么需要约定好全局的组件–数据映射关系。

> 实践建议：如果选择占位符方案，务必配合前端骨架屏。用户看到的应该是"加载中的卡片轮廓"，而不是一闪而过的占位符文本。

方案三：自定义标签（XML-like）

这个方案的灵感来自 bolt.new。bolt.new 在模型输出中引入了类 HTML 的自定义标签来描述 Action：

```
<boltArtifact title="Some title" id="artifact_1">
  <boltAction type="shell">npm install</boltAction>
```

```
</boltArtifact>
```

这种方式需要在 Markdown 解析之外单独写一个 XML parser，从文本流中提取 <boltArtifact> 标签及其属性和子元素（解析参考：https://github.com/stackblitz/bolt.new/blob/main/app/lib/runtime/message-parser.ts）。

自定义标签的核心优势在于**表达力最强**——XML 天然支持嵌套结构、属性、事件绑定（如 on-click="run"），可以描述比 JSON 更复杂的 UI 语义。而且它与模型无关，任何能生成合法 XML 的模型都可以接入，这对于需要同时对接多家模型（Qwen、Claude、GPT）的场景尤其有价值。

但它的代价也很明显：需要维护一个独立的流式 XML parser，且必须处理好与 Markdown 解析器的协作关系（哪些部分走 Markdown 渲染，哪些部分走 XML 解析）。这个方案的诞生背景是部分模型厂商未提供 Tool Call API，需要通过 Prompt 工程来模拟工具调用。如果你的模型已经支持 Function Calling，这个方案的必要性就降低了。

三种方案的选择建议

总结一下：**代码块扩展**最稳健，复用现有解析链路、零额外依赖，适合大多数场景；**占位符替换**最轻量，适合卡片数据完全由服务端提供的场景；**自定义标签**表达力最强，适合需要多模型统一管控、或已有 XML 基础设施的团队。
我们在生产中选择了代码块扩展，原因很朴素：改造成本最低、模型约束最简单、出问题时最容易排查。

但到这里只解决了"卡片怎么写进 Markdown"的问题。紧接着的问题是——卡片里的 JSON 数据从哪来？模型自己编的商品价格能用吗？库存状态、用户偏好这些实时信息怎么办？

03 卡片数据从何而来

数据与 UI 必须解耦

这是一个关键的架构决策。卡片的视觉结构（"这是一个商品卡，有标题、价格、图片"）和卡片的业务数据（"iPhone 15 Pro Max, ¥9999, 库存 32"）必须分开处理。

原因很现实：大模型的预训练数据不可能涵盖所有商品，即使涵盖了，价格库存也在实时变化，更不用说千人千面的个性化推荐了。如果让模型直接生成完整的商品数据，你会得到一个"看起来像回事但完全不能用"的 JSON。

这里我们归纳为三个阶段的方案演进。

方案一：模型直出——能用，但不可靠

最简单的方式：在 System Prompt 中告诉模型"需要展示商品时，生成一个包含完整数据的代码块"，模型直接把标题、价格、图片 URL 都填进 JSON 里。

这在 Demo 阶段确实能跑通。但一上线就暴露了问题：

- 模型编造了不存在的商品链接（胡编乱造）
- 价格和实际售价不一致（训练数据过时）
- 同一个商品对不同用户展示相同价格（无法做个性化）

结论：模型直出只适合静态内容（如百科摘要、功能说明），不适合任何涉及实时业务数据的场景。

方案二：增量 Patch 更新——先占位，后补数据

既然模型不能直接生成可靠数据，那换一个思路：**让模型只生成卡片的"骨架"（一个带 ID 的占位 JSON），服务端异步获取真实数据后通过 Patch 机制更新到前端。**

模型输出的内容是这样的——注意代码块里只有一个 `id`，没有真实数据：

````
为你推荐以下商品：
```ProductCard
{
  "id": "xxx"
}
```
这些商品不仅实用，而且价格便宜。
````

然后前端和服务端各干各的事：

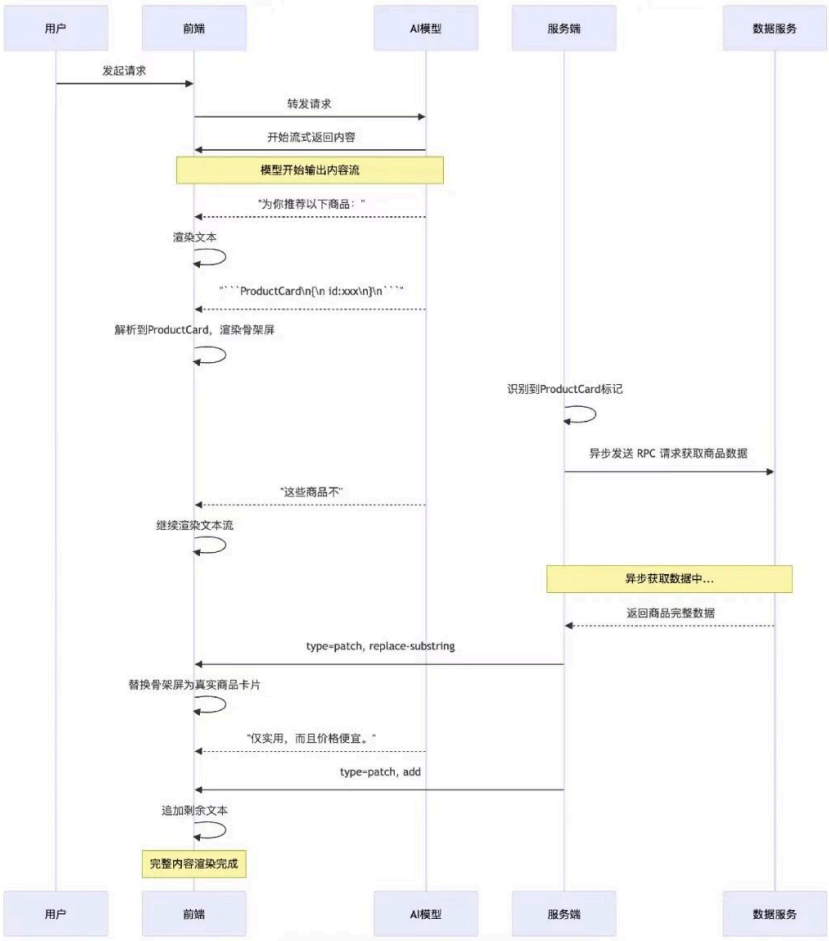
前端：解析到 `ProductCard` 代码块后，发现数据不完整，渲染一个骨架屏占位。

服务端：在识别到模型输出中包含 `ProductCard` 代码块时，立即异步发起 RPC 请求获取商品数据。数据回来后，通过我们在标准 JSON Patch 之上扩展的 `replace-substring` 操作符，将占位 JSON 替换为完整数据。

传输消息格式如下：

```
[
  {
    "type": "full",
    "data": {
      "markdown": "为你推荐以下商品：\n```ProductCard\n{\n  id:xxx\n}\n``
    }
  },
  {
    "type": "patch",
    "patch": [
      {
        "op": "replace-substring",
        "path": "/markdown",
        "substring": "```ProductCard\n{\n  id:xxx\n}\n```",
        "replacement": "```ProductCard\n{\n  \"id\": \"12345\",\n  \"t
      }
    ]
  },
  {
    "type": "patch",
    "patch": [
      {
        "op": "add",
        "path": "/markdown",
        "value": "仅实用，而且价格便宜。"
      }
    ]
  }
]
```

整个时序是这样的：



这个方案解决了数据准确性问题，但体验上有一个明显缺陷：**用户会先看到骨架屏，等数据回来后才看到真实卡片，有一个可感知的"跳变"**。如果 RPC 请求慢（比如跨地域调用），这个延迟会更明显。

另外还有一个工程复杂度问题：`replace-substring` 需要在服务端精确匹配 Markdown 文本中的占位片段再做替换，如果模型输出的格式有微小偏差（多了个空格、换行不一致），替换就会失败。这在生产中需要做鲁棒性处理。

方案三：Tool 驱动——让工具同时生产数据和 UI

方案二的核心矛盾在于"模型先返回，数据后补"，数据和 UI 在时序上是割裂的。有没有可能让它们一步到位？

答案是：**把数据获取和 UI 描述都交给 Tool 来做。** Agent 识别到用户意图后调用一个工具（比如 `search_products`），工具直接返回结构化数据 + UI 描述，前端按约定渲染。模型不再需要"编"数据，它只负责意图理解和对话编排。

社区中有两个代表性协议在探索这条路：**MCP Apps** 和 **A2UI**。它们解决的是同一个问题——Agent 如何驱动前端渲染 UI？但设计哲学截然不同。

- MCP Apps：工具驱动 UI

MCP Apps 是 Anthropic 在 MCP（Model Context Protocol）基础上的扩展。它的核心理念是：**Tool 返回的不只是纯数据，还附带一份 UI 描述。**

具体来说：

- 每个 MCP Tool 在注册时就声明"我的返回结果应该用什么 UI 来展示"
- Agent 调用 Tool 后，前端根据 `tool_id` 匹配预注册的渲染器，自动渲染卡片
- 卡片内的交互事件可以回调到 Agent，形成闭环

MCP Apps 的设计隐喻是：**UI 是 Tool 执行结果的附属产物**。工具先把活干了（查商品、查航班），然后顺便告诉前端"结果长这样"，用什么 UI 地址渲染。这意味着 UI 和 Tool 是强绑定的——一个 `search_products` 工具对应一种 `ProductList` 卡片。

- A2UI：Agent 驱动 UI

A2UI（Agent to UI） 是 Google 提出的通用声明式 UI 协议。它的设计哲学和 MCP Apps 完全不同：**不绑定任何工具链，纯粹定义"界面应该长什么样"**。

A2UI 定义了一套 JSON Schema，涵盖布局、表单、列表、图表等常见 UI 组件。Agent 只需输出符合 Schema 的 JSON，各端（Web / iOS / Android）各自实现渲染器。Action 事件通过标准事件总线回传给 Agent。

与 MCP Apps 相比，A2UI 更关注渲染层本身的标准化。它不关心数据是从 Tool来的还是模型直接生成的——它只管"拿到一份 JSON，画出对应的 UI"。

- 两者的本质区别

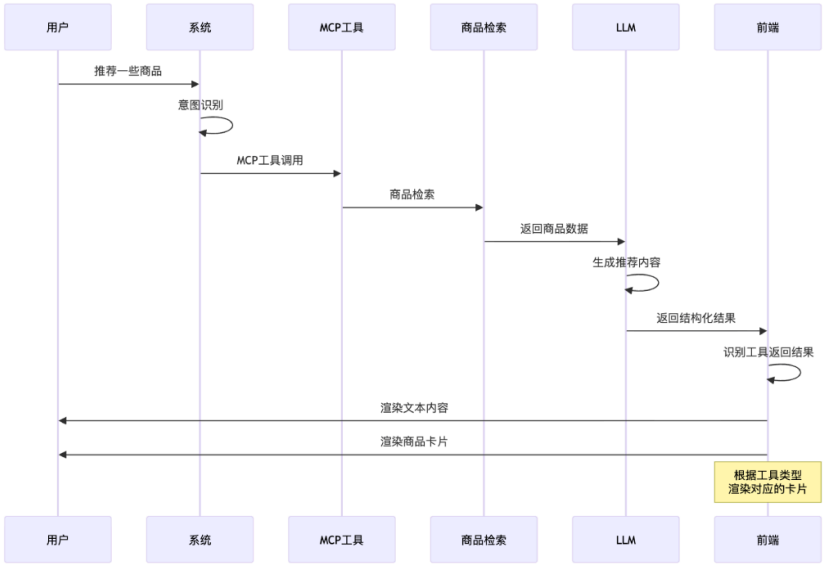
理解这两个协议的关键在于看它们的**驱动方式**：

- **MCP Apps**：工具驱动。UI 是 Tool 的附属，粒度是 Tool 级别（一个工具对应一种卡片）。适合卡片类型确定、交互模式固定的场景。
- **A2UI**：Agent 驱动。UI 由 Agent 自主组合，粒度是组件级别（可以任意拼装布局、表单、列表）。适合需要动态生成 UI、或者从预设卡片向 Agent 自主生成过渡的场景。

两者在实践中并非互斥。一种可行的组合方式是：在 MCP Tool 层使用 MCP Apps 的绑定机制来管理 Tool 与 UI 的映射关系，同时用 A2UI 的 JSON Schema 作为 UI 描述的标准格式——这样既有 Tool 层的确定性，又有 UI 层的通用性。

- Tool 驱动的整体流程

无论选择哪种协议，Tool 驱动 UI 的核心时序是一致的，我们在实践的过程中采用 Tool 返回的数据 + UI 的方式：



Tool 驱动方案的优势非常明显：

- 数据与 UI 在 Tool 层一步到位，没有"先占位后补数据"的体验断裂
- 业务方自主维护 Tool 和对应的卡片，Agent 只做编排，职责清晰
- 天然支持复杂交互——卡片内分页、筛选、表单提交都可以通过 Tool 回调实现

三种方案的演进逻辑

回顾这三种方案，它们不是简单的"好坏之分"，而是一条**渐进式的架构演进路径**：

模型直出 → 最快出活，适合 Demo 和验证阶段

增量 Patch → 数据可靠了，但时序和工程复杂度上升

Tool 驱动 → 架构最干净，但需要 MCP 工具链和协议基础设施

核心方向是：**把数据生产的责任从模型转移到工具链，让模型专注于意图理解和对话编排。**

到这里，我们讨论了两个问题：卡片怎么嵌入 Markdown 流（第一部分的三种扩展方案），以及卡片数据从哪来（模型直出 → 增量 Patch → Tool 驱动）。

但当我们要把这套方案推广到实际业务的时候，还需要考虑新的问题：A 团队用代码块扩展，B 团队用占位符；C 团队的消息格式和 D 团队不兼容；Web 端和 iOS 端对同一种卡片的事件约定完全不同。每接入一个新业务方，前端就要写一套新的解析和渲染逻辑。

这其实会让我们去思考一个更根本的问题：**如何用协议来收敛混乱？**

04 多团队协作怎么不乱——四层统一协议

为什么需要协议

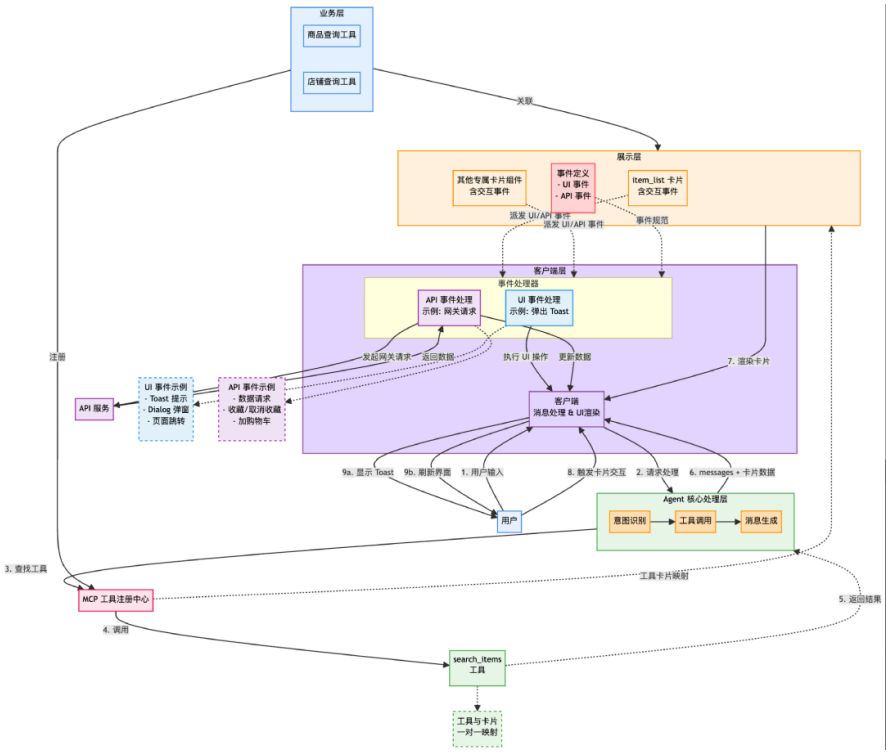
"协议"这个词听起来很重，但它要解决的问题非常朴素：**让不同团队、不同端的开发者面向同一套规范工作，减少重复建设和沟通成本。**

没有协议的时候，每个 Agent 团队都在各自定义 Markdown 扩展格式、消息传输结构、卡片事件约定。前端为每种 Agent 写一套解析器，iOS 和 Android 各自实现一遍，改一个字段要同步通知 N 个团队。这种"自由"的代价是系统迅速碎片化。

我们把协议分成四层，每一层解决一个明确的问题：

协议层	解决什么问题	一句话描述
1. Markdown 标记协议	卡片在文本流中怎么写	约定使用哪种 Markdown 扩展方式、支持的组件类型
2. 消息传输协议	前后端之间传什么格式	定义流式响应的数据包结构（全量 / 增量 / 推荐）
3. UI 渲染协议	卡片长什么样	标准 JSON Schema，Web/iOS/Android 共用一份描述
4. 事件通信协议	用户点了卡片之后怎么办	定义卡片可触发的 Action 及其响应方式

四层协议的运作关系如下图所示：



下面逐层展开。

第一层：Markdown 标记协议

回到第一部分的问题：我们列举了代码块扩展、占位符替换、自定义标签三种方案。如果不做约束，不同 Agent 团队各选各的，前端就要为每种格式维护一套解析器。

Markdown 标记协议就是做这个约束：**统一选定一种扩展方式，定义支持的组件类型列表，规范 JSON 属性的命名和格式。**

有了这层协议后：

- 所有 Agent 共用一份 System Prompt 中的"卡片生成规范"，只需调试一种输出格式
- 前端只维护一套 Markdown 解析器，新增卡片类型只需注册组件，不需要改解析逻辑
- 设计师可以提前预知每种标记渲染出来的样子，统一调整视觉规范

第二层：消息传输协议

模型输出经过 Agent 编排后，需要通过一个统一的消息格式传输给前端。我们约定每次传输的是一个"动作组"：

```
data: [
  {"type": "text", "content": "第一条消息"},
  {"type": "recommend/prompt", "content": ["你可能喜欢1", "你可能喜欢2"]}
]
```

`type` 标识动作类型：`text` 是模型返回的 Markdown 消息，`recommend/prompt` 是追问推荐。第二部分讲到的增量 Patch 更新，其中的 `type=full` 和 `type=patch` 也属于消息传输协议的一部分——正是有了统一的数据包结构，前端才能用同一套逻辑处理全量消息和增量更新。

统一消息格式之后，最大的收益是前端组件的可复用性。无论接入哪个 Agent，消息解析逻辑都是同一套代码。新业务方接入时，不需要前端配合"定制消息格式"，只要按协议传输就行。

第三层：UI 渲染协议

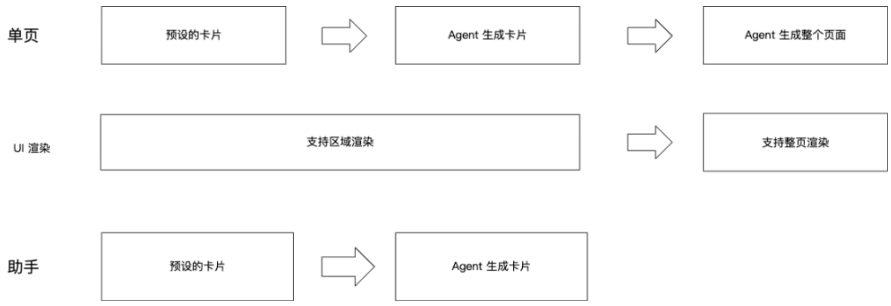
这一层定义的是：**一张卡片的结构化描述应该长什么样？**

核心理念来自第二部分介绍的 A2UI 协议——Agent 输出一份标准 JSON，各端（Web / iOS / Android）各自实现渲染器。这种"描述与渲染分离"的设计，使得同一份 JSON 可以在三端呈现一致的 UI，而不需要 Agent 关心前端的技术实现。

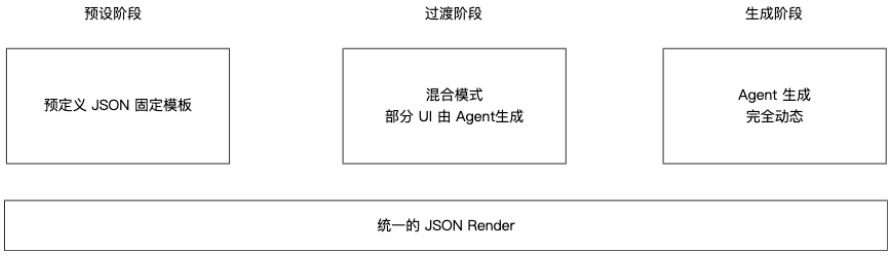
结合现状和对模型能力发展的期望， LUI（Language User Interface）的演进可以分为两个阶段：

第一阶段：预设卡片。 模型能力尚不稳定，通过预定义的卡片模板 + JSON 数据来保证产品对生成内容的把控力。每种卡片有固定的 Schema，Agent 只需要填数据。

第二阶段：Agent 生成。 随着模型能力提升，由 Agent 自主组合 UI 组件，实现千人千面的动态卡片。Agent 不再受限于预设模板，而是根据上下文自由拼装布局、表单、列表。



这两个阶段的关键在于**平滑过渡**。A2UI 协议因为采用了框架无关的 JSON Schema，预设卡片和 Agent 动态生成可以共享同一套渲染器——切换时前端零改动。所以将 A2UI 作为 UI 渲染协议虽然先阶段艰难，但是长期看会是一个不错的选择。



第四层：事件通信协议

前三层解决了"卡片怎么写、怎么传、怎么画"，但还缺一环：**用户和卡片交互之后，发生什么？**
我们的设计原则是：**卡片 JSON 只描述"是什么"和"能做什么"，不包含"怎么做"**。执行逻辑由端侧事件处理器统一承担。
这样做有两个好处：一是 Agent 生成的 JSON 不包含可执行代码，降低安全风险（想想如果 Agent 可以往卡片里注入任意 JS 代码会怎样）；二是事件派发逻辑端侧统一实现，各业务方无需重复建设。

举个具体的例子走一遍完整流程：

1. 渲染阶段：Agent 返回一个商品卡片的 JSON，其中按钮声明了一个 Action：

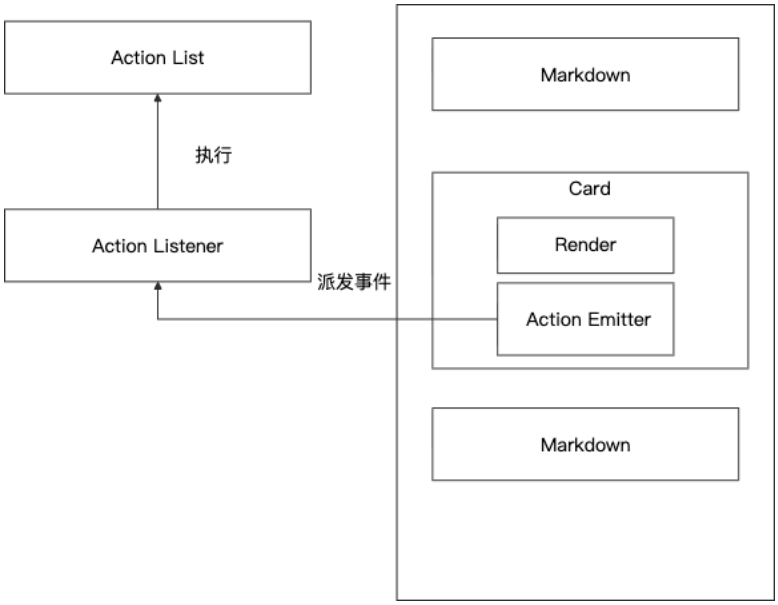
```
{
  "type": "button",
```

```
    "text": "加入购物车",
    "action": {
      "type": "api",
      "action": "addToCart",
      "params": { "itemId": "12345" }
    }
  }
```

2. 交互阶段：用户点击按钮，事件处理器接收到 Action，识别 `type=api`，发起网关请求调用加购接口。

3. 反馈阶段：请求返回后，事件处理器根据结果更新卡片状态——成功则按钮变灰显示"已加购"，失败则弹出 Toast 提示错误原因。

整个过程中，卡片 JSON 是"纯声明"的——它说"我有一个按钮，点了之后应该调 addToCart"，但具体怎么调、调完怎么更新 UI，全部由端侧事件处理器负责。



A2UI 本身已经定义了 Action 规范，在此基础上需要做的工作是：梳理业务中实际需要的事件类型（UI 事件：Toast、Dialog、页面跳转；API 事件：数据请求、收藏、加购），定义统一的事件格式和消费方式，让各业务方不需要自己发明事件协议，如果是 H5 那就是 uniapi。

05 总结

卡片式交互不是"在聊天框里塞组件"这么简单。它重新定义了 Agent 时代的前后端协作方式：

- 模型从"文本生成器"变成"界面规划师"——它需要理解什么时候该展示卡片、展示什么类型的卡片；
- 后端从"数据提供者"变成"协议协调者"——它需要在 Tool 层打通数据获取与 UI 描述；

- 前端从"页面渲染器"变成"协议执行引擎"——它面向标准 JSON Schema 渲染，而非为每个业务方定制。

四层协议体系（Markdown 标记 → 消息传输 → UI 渲染 → 事件通信）的价值，不在于技术有多先进，而在于为复杂系统建立了**确定性**——当每一层的输入输出都有明确约定，问题排查、能力复用、新业务接入的效率就会指数级提升。

最后，值得关注的是社区中正在涌现的 Agentic 协议生态：

- **AG-UI**（Agent-用户交互协议）：基于事件的标准，连接 Agent 与面向用户的应用
- **MCP**（Anthropic）：Agent 安全连接外部工具、工作流和数据源的开放标准
- **A2A**（Google）：分布式 Agent 系统中 Agent 之间的协调协议
- **A2UI**（Google）/ **MCP Apps**（Anthropic）/ **Open-JSON-UI**（OpenAI）：声明式 UI 规范，定义 Agent 响应的视觉组织方式
- **Web MCP**：让网页本身作为 MCP Server，为 Agent 提供工具访问

卡片式对话只是 Agentic 交互的起点。当协议足够成熟、模型能力持续提升，对话界面将真正成为用户与数字世界交互的统一入口。希望在真实的业务落地之后，能够有更多的分享。

如果你也在探索类似方向，欢迎在评论区交流——协议设计、渲染器实现、Prompt 工程、MCP 工具封装，任何话题都行。

06 参考资料

- https://github.com/stackblitz/bolt.new
- https://modelcontextprotocol.io/extensions/apps/overview
- https://a2ui.org/
- https://docs.copilotkit.ai/built-in-agent/generative-ui/a2ui
- https://remarkjs.github.io/react-markdown/

07 团队介绍

本文作者无二，来自淘天集团-跨端技术团队。本团队服务于淘宝基础用户产品，是淘宝重要的业务线之一。团队以前端、Weex、Native端的技术解决方案框架和研发模式不断完善自己，持续探索端智能等创新，打造极致的体验和工程技术，保障多端设备的适配和稳定运行，致力于让亿级规模的交付能够更丝滑、更稳定。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 7830

留言

写留言

还没有留言，来写首评